

User Guide: LinkedIn Job Finder

Documentation of Python Scripts, Search Parameters, AI-Assisted Workflow, and File Export

Author: Javier Medina Moreno (2026) | License: GNU AGPLv3

General Summary: LinkedIn Job Finder is an automated data acquisition suite of four modular Python scripts designed to streamline professional job hunting. It manages the complete search lifecycle: discovering matching vacancies across public guest endpoints (`Finder.py`), extracting comprehensive role specifications (`Extractor.py`), partitioning dossiers into token-friendly batch chunks (`Splitter.py`), and assembling high-priority opportunities into a clean, renumbered application dossier (`Filter.py`). All network interactions query LinkedIn's guest pages directly without requiring accounts, login credentials, or private API keys, while atomic disk writes guarantee zero loss of previously collected data.

TABLE OF CONTENTS

1. What is LinkedIn Job Finder?	1
2. How the Workflow Operates	2
3. Internal Code Structure	3
4. Generated Files and Data Flow	4
5. Step-by-Step User Guide	5
6. Authorship and License of Use	5

1. What is LinkedIn Job Finder?

It is an engineering and data acquisition framework developed to eliminate the manual friction of scouting professional vacancies. Rather than spending hours clicking through paginated web cards, opening dozens of ephemeral browser tabs, and copying text fragments manually, this pipeline executes systematic vacancy queries directly against LinkedIn's public guest infrastructure.

By bypassing heavy headless browser environments (such as Selenium or Playwright) in favor of optimized HTTP sessions, the tool provides high throughput, deterministic execution, and complete isolation from your personal LinkedIn profile. Output dossiers are automatically sanitized, structured, and segmented for direct evaluation with Large Language Models (LLMs) to perform automated candidate-to-job matching at scale.

2. How the Workflow Operates

2.1. Discovering Job Postings (Finder.py)

- **Boolean Keyword Matching:** Evaluates complex logical expressions (e.g., combining academic degrees and core skills like `Economics`, `Python`, or `Power BI`) paired with target geographic regions.
- **Native URL Parameter Mapping:** Automatically translates search parameters into native LinkedIn guest query filters: publication timeframe (`TIME_FILTER`), experience seniority (`EXPERIENCE_LEVEL`), workplace arrangement (`WORKPLACE_TYPE`), and contract type (`JOB_TYPE`).
- **Opportunistic Pagination:** Traverses 10-job search slices up to LinkedIn's public guest ceiling of 1,000 listings, terminating gracefully when listings are exhausted or consecutive duplicate batches appear.
- **Atomic Disk Persistence:** Writes every newly discovered position immediately to the CSV file with explicit write-flushing, guaranteeing zero data loss if execution is paused or interrupted.

2.2. Extracting Full Descriptions (Extractor.py)

- **Full Specification Retrieval:** Search result cards only display truncated summaries; Extractor loads individual posting endpoints in the background to retrieve complete responsibilities, qualifications, company metadata, and publication dates.
- **DOM Sanitization & Formatting:** Strips interactive web UI toggles ("Show more" / "Show less"), extracts core metadata badges, and normalizes unstructured descriptions into clean, readable paragraphs.
- **Resumable State Auditing:** Scans the destination text dossier prior to issuing network requests, skipping already-extracted vacancies to conserve bandwidth and prevent duplicate entries.
- **Autonomous Chaining:** Operates either as an automated handoff initiated directly upon Finder completion, or as an independent utility processing targeted CSV files and URL arrays.

2.3. Batch Splitting & AI Curation (Splitter.py & Filter.py)

- **Context-Aware Batch Chunking:** Massive text dossiers easily overflow LLM prompt context limits; Splitter divides large dossiers into uniform batches of N positions (default: 50 jobs per batch).
- **Multi-Batch Prompt Screening:** Batches are uploaded alongside the user's CV to an advanced LLM to screen opportunities across career growth, acceptance probability, and earning potential.
- **Deduplicated Dossier Assembly:** Filter parses the numeric Job IDs selected by the AI, sequentially rennumbers header tags (Job #1, Job #2, etc.), and compiles an uncluttered master list of active target applications.

3. Internal Code Structure

The pipeline decouples high-speed exploratory discovery from deep specification parsing across four specialized Python modules:

Script & Function	Purpose and Role in the Workflow
Finder.py: User Input Fields	Configures Boolean query keywords, location, date filters, experience level, and extractor handoff settings.
Finder.py: scrape_linkedin_jobs()	Paginates through public guest listings, writes new jobs to the CSV atomically, and manages handoff.
Extractor.py: normalize_job_url()	Extracts canonical 8+ digit numeric IDs via regex and standardizes arbitrary links into clean URLs.
Extractor.py: get_existing_job_ids()	Reads destination text files to construct an in-memory set of completed Job IDs for safe, resumable runs.
Extractor.py: parse_linkedin_job_url()	Fetches public detail endpoints, sanitizes DOM markup via BeautifulSoup, and manages HTTP 429 throttling back-off.
Extractor.py: parse_multiple_job_urls()	Orchestrates bulk extraction sequentially, applies stochastic politeness delays, and cleanly handles interrupts.
Splitter.py: split_job_details()	Divides large text dossiers into standalone batch files of 50 jobs each, preserving structured boundary dividers.
Filter.py: parse_source_job_blocks()	Indexes the master text dossier into regex-bounded blocks accessible directly by their numeric Job ID.
Filter.py: filter_interesting_jobs()	Extracts selected positions, renumbers headers sequentially (Job #n), and saves them to the curated file.

4. Generated Files and Data Flow

Data moves across four structured persistence layers designed to maintain modularity and state safety throughout execution:

4.1. Search Results Spreadsheet (`Finder.py` → `linkedin_jobs.csv`)

Description: A standardized CSV catalog consolidating primary vacancy records discovered during search pagination.

Key elements: Clean tabular schema (`job_id`, `title`, `company`, `location`, `date_posted`, `url`) with automatic deduplication.

4.2. Master Details Dossier (`Extractor.py` → `linkedin_job_details.txt`)

Description: A comprehensive text file storing full role specifications, responsibilities, and qualifications for every vacancy.

Key elements: 80-character boundary dividers (`=====`), clean paragraph structure, stripped UI button artifacts, and sequential numbering.

4.3. Partitioned Batch Segments (`Splitter.py` → `linkedin_job_details_batch_*.txt`)

Description: Standalone text chunks partitioned from the master dossier, sized specifically for LLM prompt context injection.

Key elements: Configurable batch sizing (default: 50 jobs per chunk) with range-indexed filenames (e.g. `batch_1_jobs_1-50.txt`).

4.4. Final Curated List (`Filter.py` → `interesting_jobs.txt`)

Description: A targeted text file containing only the high-priority positions selected by the AI evaluation as viable opportunities.

Key elements: Renumbers selected jobs consecutively (Job #1, Job #2, etc.) for clean reading without modifying original specifications.

4.5. Built-in Politeness and Safety Controls

Description: Multi-tiered operational protections that keep scraping sustainable, polite, and indistinguishable from regular browsing.

Key elements: Randomized pauses between requests (2.5 to 5.5s), automatic 35-second cooldown upon HTTP 429 (Too Many Requests), and clean interruption via Ctrl+C.

5. Step-by-Step User Guide

1. Set Up Your Search (Finder.py):

Open `Finder.py` and configure parameters in `USER INPUT FIELDS`:

- **Keywords:** `KEYWORD` (e.g. `'(Economics OR Business) AND ("Power BI" OR Python)'`).
- **Location:** `LOCATION` (e.g. `'Barcelona, Catalonia, Spain'` or `'Remote'`).
- **Filters:** `TIME_FILTER` (`'r604800'` for past week), `EXPERIENCE_LEVEL` (`'1,2'` for internship and entry-level).
- **Automated Extraction:** Set `AUTO_RUN_EXTRACTOR = True` for automated chaining.

2. Run Discovery Search:

Open your terminal and run `python Finder.py`. Discovered postings append immediately to `linkedin_jobs.csv`. If enabled, Extractor launches automatically upon completion.

3. Download Full Descriptions (Extractor.py):

To extract manually or target specific URLs, define `TARGET_URLS` or keep `INPUT_CSV = 'linkedin_jobs.csv'`, then execute `python Extractor.py`. Existing IDs in `linkedin_job_details.txt` are automatically skipped.

4. Split Dossier for LLM Ingestion (Splitter.py):

To prevent context overflows, verify `BATCH_SIZE = 50` in `Splitter.py` and execute `python Splitter.py`. This creates discrete chunk files (e.g. `batch_1_jobs_1-50.txt`) in your working folder.

5. Screen Jobs with an AI LLM Model (Recommended: Gemini Flash with Thinking):

Upload your CV alongside as many batch files as can fit comfortably within the context window of your AI model of choice. Write a prompt indicating your current situation, background constraints, and target career goals, specifying the exact assignment syntax so the output can be pasted directly into `Filter.py`.

Example Prompt:

```
This is my CV. I lean towards business strategic analysis roles for the long term,
but I'm a recent graduate without experience that has to start from the bottom.
From the given list of jobs, select all IDs of jobs that could fit me. Consider:
1. My chances of being accepted.
2. My career development prospects.
3. My future earnings potential.

Show IDs like this:

INTERESTING_JOB_IDS = """
4458756589
4426760640
4411647731
4409222002
"""
```

6. Assemble Curated Application Dossier (Filter.py):

Copy the `INTERESTING_JOB_IDS = """..."""` block returned by the AI into `Filter.py` and run `python Filter.py`. The script extracts the matching vacancies, rennumbers headers sequentially (Job #1, Job #2, etc.), and generates your clean `interesting_jobs.txt` shortlist.

6. Authorship and License of Use

Author: Javier Medina Moreno (2026)

License: GNU AGPLv3 (GNU Affero General Public License v3.0). Free to use, modify, and redistribute under the terms of the GNU AGPLv3. All derivative works and modifications must retain original author attribution and copyright notices.